

PS4

Aishani Mathur

2001-02-07

Due 02/07 at 5:00PM Central.

“This submission is my work alone and complies with the 30538 integrity policy.” Add your initials to indicate your agreement: AM

Github Classroom Assignment Setup and Submission Instructions

1. Accepting and Setting up the PS4 Assignment Repository

- Each student must individually accept the repository for the problem set from Github Classroom (“ps4”) – <https://classroom.github.com/a/hWhcHqH>
 - You will be prompted to select your cnetid from the list in order to link your Github account to your cnetid.
 - If you can’t find your cnetid in the link above, click “continue to next step” and accept the assignment, then add your name, cnetid, and Github account to this Google Sheet and we will manually link it: <https://rb.gy/9u7fb6>
- If you authenticated and linked your Github account to your device, you should be able to clone your PS4 assignment repository locally.
- Contents of PS4 assignment repository:
 - `ps4_template.qmd`: this is the Quarto file with the template for the problem set. You will write your answers to the problem set here.

2. Submission Process:

- Knit your completed solution `ps4.qmd` as a pdf `ps4.pdf`.
 - Your submission does not need runnable code. Instead, you will tell us either what code you ran or what output you got.
- To submit, push `ps4.qmd` and `ps4.pdf` to your PS4 assignment repository. Confirm on Github.com that your work was successfully pushed.

Grading

- You will be graded on what was last pushed to your PS4 assignment repository before the assignment deadline
- Problem sets will be graded for completion as: {missing (0%); - (incomplete, 50%); + (excellent, 100%)}
 - The percent values assigned to each problem denote how long we estimate the problem will take as a share of total time spent on the problem set, not the points they are associated with.
- In order for your submission to be considered complete, you need to push both your `ps4.qmd` and `ps4.pdf` to your repository. Submissions that do not include both files will automatically receive 50% credit.

```

import pandas as pd
import altair as alt
import time

import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")

```

```

RendererRegistry.enable('png')

```

Step 1: Develop initial scraper and crawler

```

import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://oig.hhs.gov/fraud/enforcement/"
html = requests.get(url, headers={"User-Agent": "Mozilla/5.0"}).text
soup = BeautifulSoup(html, "html.parser")

cards = soup.select("li.usa-card")
rows = []

for card in cards:
    a = card.select_one("h2.usa-card__heading a")
    if not a:
        continue

    title = a.get_text(strip=True)
    link = "https://oig.hhs.gov" + a["href"]

    date_tag = card.select_one("span.text-base-dark.padding-right-105")
    date = date_tag.get_text(strip=True) if date_tag else None

    cat_tag = card.select_one("ul.display-inline.add-list-reset li")
    category = cat_tag.get_text(strip=True) if cat_tag else None

    rows.append({
        "title": title,
        "date": date,
        "category": category,
    })

```

```

        "link": link
    })

df = pd.DataFrame(rows)
df.head()

```

	title	date	category	link
0	Houston Transplant Doctor Indicted For Making ...	February 5, 2026	Criminal and Civil Actions	htt
1	MultiCare Health System to Pay Millions to Set...	February 4, 2026	Criminal and Civil Actions	htt
2	Brooklyn Banker Pleads Guilty to Laundering Pr...	February 3, 2026	COVID-19	htt
3	Delafield Man Sentenced to 18 Months' Imprison...	February 3, 2026	Criminal and Civil Actions	htt
4	Former NFL Player Convicted for \$197M Medicare...	February 3, 2026	Criminal and Civil Actions	htt

HTML inspection showed that each enforcement action is contained within an li tag with class usa-card, with the title in an h2 heading, the date in a span element, and the category listed within an inline unordered list - ul tag.

Step 2: Making the scraper dynamic

1. Turning the scraper into a function

- a. Pseudo-Code Define function scrape_enforcement_actions(start_year, start_month, run_scraper)

If start_year < 2013, print warning message and exit function

If run_scraper is False, exit function without scraping

Create start_date using start_year and start_month

Initialize empty list rows

Set page = 1

While True:

Build URL for current page using page number

Request webpage and parse HTML

Select all enforcement action cards on the page

If no cards are found, break loop

For each card on the page:

Extract title and link

Extract date and convert to datetime

Extract category

If date is on or after start__date, add row to rows

Keep track of dates seen on this page

If the oldest date on the page is earlier than start__date, break loop

Increment page by 1

Pause for 1 second

Convert rows into a dataframe

Save dataframe as enforcement__actions__year__month.csv

Return dataframe

- b. Create Dynamic Scraper

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

def scrape_enforcement_actions(start_year: int,
                               start_month: int,
                               run_scraper: bool = True) -> pd.DataFrame |
↳ None:
    if start_year < 2013:
        print("Please restrict to year >= 2013 (only actions after 2013 are
↳ listed).")
        return None

    if not run_scraper:
        print("run_scraper=False, skipping web scraping.")
        return None

    start_date = pd.Timestamp(start_year, start_month, 1)

    base_url = "https://oig.hhs.gov/fraud/enforcement/"
    headers = {"User-Agent": "Mozilla/5.0"}

    rows = []
```

```

page = 1

while True:
    page_url = base_url if page == 1 else f"{base_url}?page={page}"
    resp = requests.get(page_url, headers=headers, timeout=30)
    soup = BeautifulSoup(resp.text, "html.parser")

    cards = soup.select("li.usa-card")
    if not cards:
        break

    page_dates = []

    for card in cards:
        a = card.select_one("h2.usa-card__heading
↪ a[href~='/fraud/enforcement/']")
        if not a:
            continue

        title = a.get_text(strip=True)
        link = "https://oig.hhs.gov" + a["href"]
        date_tag =
↪ card.select_one("span.text-base-dark.padding-right-105")
        date = date_tag.get_text(strip=True) if date_tag else None
        date_dt = pd.to_datetime(date, format="%B %d, %Y",
↪ errors="coerce") if date else pd.NaT
        if pd.isna(date_dt):
            continue
        cat_tag = card.select_one("ul.display-inline.add-list-reset li")
        category = cat_tag.get_text(strip=True) if cat_tag else None

        page_dates.append(date_dt)

        if date_dt >= start_date:
            rows.append({"title": title, "date": date, "category":
↪ category, "link": link})
            if page_dates and min(page_dates) < start_date:
                break

    page += 1
    time.sleep(1)

```

```

df = pd.DataFrame(rows)

required_name = f"enforcement_actions_{start_year}_{start_month:02d}.csv"
df.to_csv(required_name, index=False)

print(f"Saved {len(df)} rows to {required_name}")
return df

```

```

#RUN_SCRAPER = False
#df_actions = scrape_enforcement_actions(2024, 1, run_scraper=RUN_SCRAPER)

```

```

df_actions = pd.read_csv("enforcement_actions_2024_01.csv")

df_actions.head()
len(df_actions)

df_actions["date_dt"] = pd.to_datetime(df_actions["date"])
earliest = df_actions.sort_values("date_dt").iloc[0]
earliest

```

```

title      Former Nurse Aide Indicted In Death Of Clarksv...
date              January 3, 2024
category              State Enforcement Agencies
link      https://oig.hhs.gov/fraud/enforcement/former-n...
date_dt              2024-01-03 00:00:00
Name: 1786, dtype: object

```

Starting from January 2024, the dynamic scraper collected 1787 enforcement actions in total. The earliest was dated January 3, 2024 and titled “Former Nurse Aide Indicted in Death of Clarksville ...”, which falls under the State Enforcement Agencies category.

- c. Test Your Code

```

#RUN_SCRAPER = False
#df_2022 = scrape_enforcement_actions(2022, 1, run_scraper=RUN_SCRAPER)

```

```

df_2022 = pd.read_csv("enforcement_actions_2022_01.csv")

```

```

n_actions = len(df_2022)
n_actions

```

3377

```
df_2022["date_dt"] = pd.to_datetime(df_2022["date"])
earliest = df_2022.sort_values("date_dt").iloc[0]
earliest
```

```
title      Integrated Pain Management Medical Group Agree...
date              January 4, 2022
category              Fraud Self-Disclosures
link      https://oig.hhs.gov/fraud/enforcement/integrat...
date_dt              2022-01-04 00:00:00
Name: 3376, dtype: object
```

Starting from January 2022, the dynamic scraper collected 3,377 enforcement actions in total. The earliest enforcement action scraped was dated January 4, 2022 and titled “Integrated Pain Management Medical Group Agree...”, which falls under the Fraud Self-Disclosures category.

Step 3: Plot data based on scraped data

1. Plot the number of enforcement actions over time

```
import pandas as pd
import altair as alt

df = pd.read_csv("enforcement_actions_2022_01.csv")
```

```
df["date_dt"] = pd.to_datetime(df["date"])

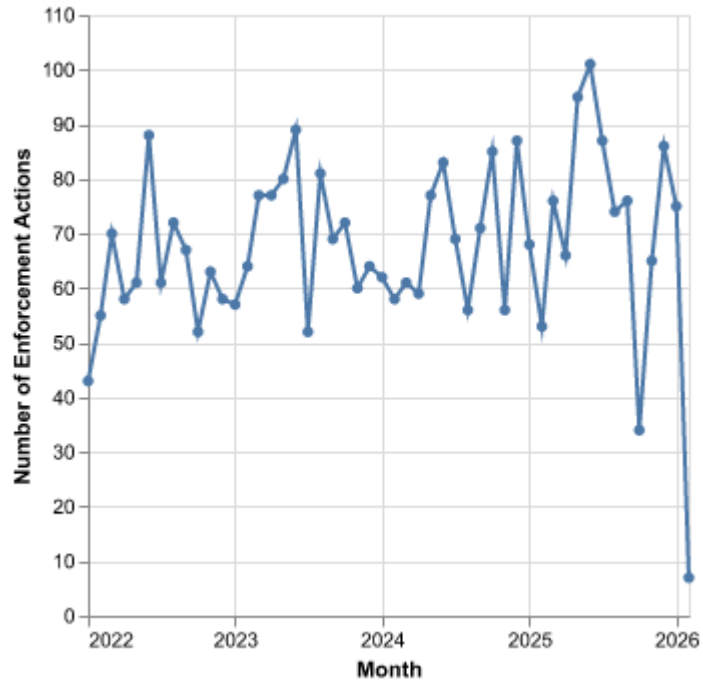
monthly_counts = (
    df
    .assign(month_year=df["date_dt"].dt.to_period("M").dt.to_timestamp())
    .groupby("month_year")
    .size()
    .reset_index(name="n_actions")
)
```

```
alt.Chart(monthly_counts).mark_line(point=True).encode(
    alt.X("month_year:T", title="Month"),
    alt.Y("n_actions:Q", title="Number of Enforcement Actions"),
).properties()
```



```
title="HHS OIG Enforcement Actions Over Time (Since January 2022)"
)
```

HHS OIG Enforcement Actions Over Time (Since January 2022)



2. Plot the number of enforcement actions categorized:

- based on “Criminal and Civil Actions” vs. “State Enforcement Agencies”

```
import pandas as pd
import altair as alt

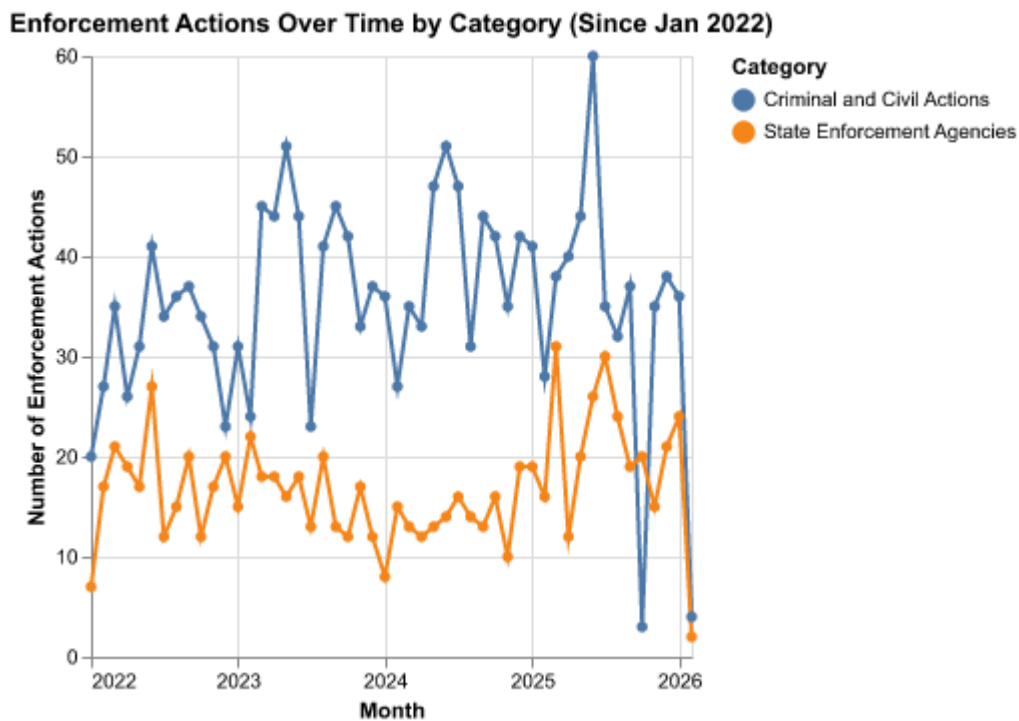
df = pd.read_csv("enforcement_actions_2022_01.csv")
df["date_dt"] = pd.to_datetime(df["date"])
df["month_year"] = df["date_dt"].dt.to_period("M").dt.to_timestamp()
cat_df = df[df["category"].isin([
    "Criminal and Civil Actions",
    "State Enforcement Agencies"
])]
cat_monthly = (
    cat_df
    .groupby(["month_year", "category"])
```

```

        .size()
        .reset_index(name="n_actions")
    )

    alt.Chart(cat_monthly).mark_line(point=True).encode(
        alt.X("month_year:T", title="Month"),
        alt.Y("n_actions:Q", title="Number of Enforcement Actions"),
        color=alt.Color("category:N", title="Category")
    ).properties(
        title="Enforcement Actions Over Time by Category (Since Jan 2022)"
    )

```



- based on five topics

```

cca = df[df["category"] == "Criminal and Civil Actions"].copy()
def classify_topic(title):
    t = title.lower()

    if any(k in t for k in ["health", "medicare", "medicaid", "hospital",
        ↪ "physician", "doctor", "clinic"]):

```

```

        return "Health Care Fraud"
    elif any(k in t for k in ["bank", "financial", "loan", "wire fraud",
        ↪ "tax", "money laundering"]):
        return "Financial Fraud"
    elif any(k in t for k in ["drug", "opioid", "controlled substance",
        ↪ "pharmacy"]):
        return "Drug Enforcement"
    elif any(k in t for k in ["bribe", "bribery", "kickback", "corruption"]):
        return "Bribery/Corruption"
    else:
        return "Other"

cca["topic"] = cca["title"].apply(classify_topic)
topic_monthly = (
    cca
    .groupby(["month_year", "topic"])
    .size()
    .reset_index(name="n_actions")
)

alt.Chart(topic_monthly).mark_line(point=False).encode(
    alt.X("month_year:T", title="Month"),
    alt.Y("n_actions:Q", title="Number of Enforcement Actions"),
    color=alt.Color("topic:N", title="Topic")
).properties(
    title="Criminal & Civil Enforcement Actions by Topic (Since Jan 2022)"
)

```

Criminal & Civil Enforcement Actions by Topic (Since Jan 2022)

